Laboratório Funcional · Aula 12

Victor Hugo Pacchioni Sgobbi · RA 240007322 · Turma C
Gerado em 16/10/2026, 19:40:44 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

Escolhi sorteia() e poe(lista, x).

sorteia() não é pura porque não tem entrada e, a cada chamada, pode devolver um valor diferente (1 a 6). Uma função pura precisa devolver sempre o mesmo resultado para os mesmos argumentos, e aqui isso não acontece.

poe(lista, x) não é pura porque modifica a lista original com append. Isso é um efeito colateral: a função altera um objeto que existe fora dela. Uma versão pura criaria e devolveria uma nova lista, sem mexer na original.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linhas 1, 4 e 7 (antes de executar):

- **Linha 1:** `proximo_id("A")` é chamada duas vezes e `contador` vai de 0 para 1 e depois para 2. Saída: `1: A-1 A-2`
- **Linha 4:** `compor(f, g)(4)` calcula `f(g(4))`. Primeiro `g(4) = 3 * 4 = 12`, depois `f(12) = 12 + 2 = 14`. Saída: `4: 14`
- **Linha 7:** `sorted(notas)` cria uma lista nova e ordenada, sem mexer em `notas`, que continua `[7, 3, 9]`. Saída: `7: [7, 3, 9] [3, 7, 9]`

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

proximo_id("A") usa a variável global contador, que aumenta a cada chamada. A mesma entrada "A" gera A-1, depois A-2, porque o resultado depende do histórico de chamadas, e não só do argumento. Isso dificulta os testes: não dá para afirmar “com "A" a saída é X”, porque o resultado depende de quantas vezes a função já foi chamada e da ordem dos testes. Para testar, seria preciso resetar o estado antes de cada teste, e os testes ficam frágeis e dependentes uns dos outros. Uma função pura como cubo(2) é sempre previsível e testável isoladamente.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Os dois cálculos:

compor(f, g)(4) = f(g(4)): primeiro g(4) = 3 × 4 = 12, depois f(12) = 12 + 2 = 14.
compor(g, f)(4) = g(f(4)): primeiro f(4) = 4 + 2 = 6, depois g(6) = 3 × 6 = 18.

A composição não é comutativa: a ordem em que as funções são aplicadas muda o resultado. Em compor(f, g), a função da direita (g) é aplicada primeiro.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e devolve uma nova lista ordenada. A lista notas continua [7, 3, 9].
notas.sort() ordena a lista no lugar, alterando o objeto original, e devolve None.

O estilo funcional é o sorted(notas). Ele não tem efeitos colaterais: o dado de entrada não muda, e o resultado vem de uma expressão que pode ser composta com outras (map, reduce, etc.). Já notas.sort() altera um estado externo, o que pode causar bugs se outra parte do programa usar notas depois. Um erro comum com sort() é escrever x = notas.sort(), o que faz x receber None.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
```

```
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de QUOTE. Ele impede a avaliação: '(A B C) é tratado como dado, uma lista de símbolos, e não como uma chamada de função. Sem ele, o Scheme tenta avaliar A, B e C como variáveis. Como nenhuma existe, ocorre o erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

não respondido

Saída do Scheme

não respondido

Saída do Python (as últimas linhas bastam)

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, a expressão cresce a cada passo: (* n (* n-1 ...)) fica com uma multiplicação pendente por chamada, e o simulador mostra isso na coluna da esquerda. Na recursão em cauda, o parâmetro parcial guarda o resultado acumulado, então a expressão não cresce: cada passo só substitui o valor anterior pelo novo. Por isso o número de pendências cresce na primeira e fica em zero na segunda.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme fez um milhão de chamadas porque a chamada conta-ate está na posição de cauda. O interpretador pode reaproveitar o registro de ativação, então a pilha não cresce. O Python não faz essa otimização: cada chamada cria um novo quadro na pilha, e o limite de profundidade (cerca de 1000) é atingido antes de um milhão de chamadas, gerando RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (confere nome obtido esperado)
  (display nome) (display ": ") (display obtido)
  (if (equal? obtido esperado)
      (display "  ok")
      (begin (display "  (esperado ") (display esperado) (display ")")))
  (newline))

(define (tamanho lis)
  (if (null? lis) 0 (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis) 0 (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))

(confere "tamanho" (tamanho '(a b c)) 3)
(confere "soma-quadrados" (soma-quadrados '(1 2 3)) 14)
(confere "pares" (pares '(1 2 3 4 5 6)) '(2 4 6))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

Em cada função, null? testa o fim da lista. car pega o elemento atual e cdr pega o resto, que é passado para a chamada recursiva. Em tamanho, car não aparece, porque só importa quantos elementos existem. Em soma-quadrados, car aparece duas vezes, para o quadrado. Em pares, car decide se o elemento entra, e cons o coloca na frente do resultado.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

(factors 28 dá os divisores de 28 até 14: 1, 2, 4, 7, 14.)

Saídas (original e com member 17)

```
2432902008176640000
[1,2,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell é não estrito: take 5 squares só calcula os primeiros cinco elementos, então a lista infinita nunca é construída por inteiro. primeiro 42 (div 1 0) não dá erro porque primeiro ignora o segundo parâmetro, e o Haskell só avalia o que é necessário. O erro de divisão só aconteceria se o valor fosse usado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Com member 17 squares, a busca percorre 0, 1, 4, 9, 16, 25, ... e nunca encontra 17, porque 17 não é quadrado perfeito. Como a lista é infinita, a função nunca termina. Já member 16 acha 16 e para. A correção é parar ao passar do valor procurado, já que a lista está em ordem crescente:

```
member b [] = False
member b (a:x)
 | a == b = True
 | a > b  = False
 | otherwise = member b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

?

Saída do JavaScript

```
15
[ 2, 5, 8 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
7
3
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

somador(10) devolve uma função que captura o parâmetro n do ambiente onde foi criada. Mesmo depois que somador termina, essa ligação continua viva, e a função devolvida ainda enxerga n = 10. Isso é um fechamento (closure).

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

A composição não é comutativa. f.compose(g) aplica g primeiro, então corresponde a compor(f, g) da Prática 1: f(g(4)) = 14. f.andThen(g) aplica f primeiro (g(f(4)) = 18), o que corresponde a compor(g, f).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
```

```
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
console.log(alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase())
  .join(", "));
```

Saídas das suas versões

```
1. 244

2. ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é a mais fácil de ler para quem conhece filter, map e sum, porque o código descreve o que se quer, não como fazer. A versão com laço é a mais fácil de depurar: dá para colocar um println ou console.log dentro do for e ver cada valor intermediário. No pipeline, um valor intermediário só aparece se você quebrar a cadeia em variáveis ou usar peek (Java) ou console.log dentro de um map.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Falso

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão com sum e map é mais curta e lê da direita para a esquerda, mas exige que quem lê conheça as formas funcionais. O livro estima cerca de 2 vezes mais tempo de execução (p. 678). Eu aceitaria esse custo em sistemas em que clareza e testabilidade pesam mais, como processamento de dados em lote; não o aceitaria em sistemas de tempo real ou com restrição forte de desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Ao colar o quicksort do livro, o erro está em [b | b <- t, b <- h]: a segunda condição deveria ser um filtro b <= h, e não outro <-. O compilador reclama de um tipo inesperado (ele tenta tratar h como lista) e mostra a inferência de tipos falhando. A correção é b <= h.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

Ao colar o quicksort do livro, o erro está em [b | b <- t, b <- h]: a segunda condição deveria ser um filtro b <= h, e não outro <-. O compilador reclama de um tipo inesperado (ele tenta tratar h como lista) e mostra a inferência de tipos falhando. A correção é b <= h.